

第一章 软件工程概述

芦效峰



软件工程概述

本章要点

- 一、软件
- 二、软件危机
- 三、软件工程
- 四、软件安全

教材：软件工程与安全

1.1软件的概念及特点

- 从1946年第一台计算机诞生，计算机已经进入各个领域、各个行业，进入每个家庭，成为现代社会必不可少的工具。
- 计算机强大功能主要依赖于计算机软件
- 软件开发就涉及到了各行各业



计算机程序

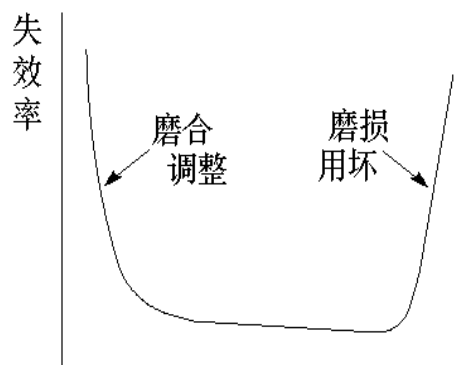
- 计算机的工作是用程序、软件来控制的



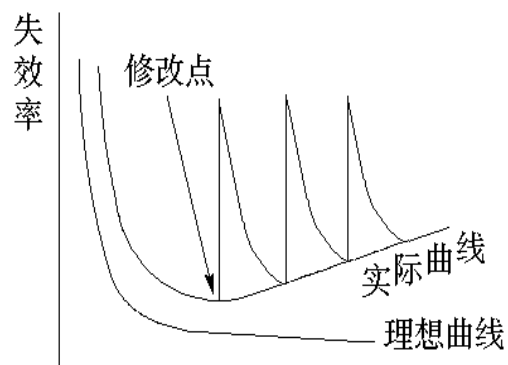
- IEEE定义：软件是计算机程序、处理和与计算机系统运行相关的文档和数据。
- 程序是按事先设计的功能和性能要求执行的指令序列
- 数据是程序运行的基础和操作的对象
- 文档是与程序开发，维护和使用有关的图文材料

2. 软件的特点

- 1. 软件是一种**逻辑实体**，而不是具体的物理实体。具有抽象性，不可描述性
- 2. 在软件的运行和使用期间，**没有硬件那样的机械磨损，老化问题**。但存在退化问题。（用户体验）



(a) 硬件失效率曲线



(b) 软件失效率曲线

2. 软件的特点

- 3. 软件的生产与硬件不同，在它的开发过程中没有明显的制造过程，而是研制。是程序员脑力劳动的结果
- 4. 可变性，软件在生产过程中甚至在投入运行之后，也还可以再改变。
- 5. 软件必须和运行软件的硬件保持一致
- 6. 软件的运行受计算机系统的影响
- 7. 软件一旦研制开发成功，其生产过程就变成复制过程

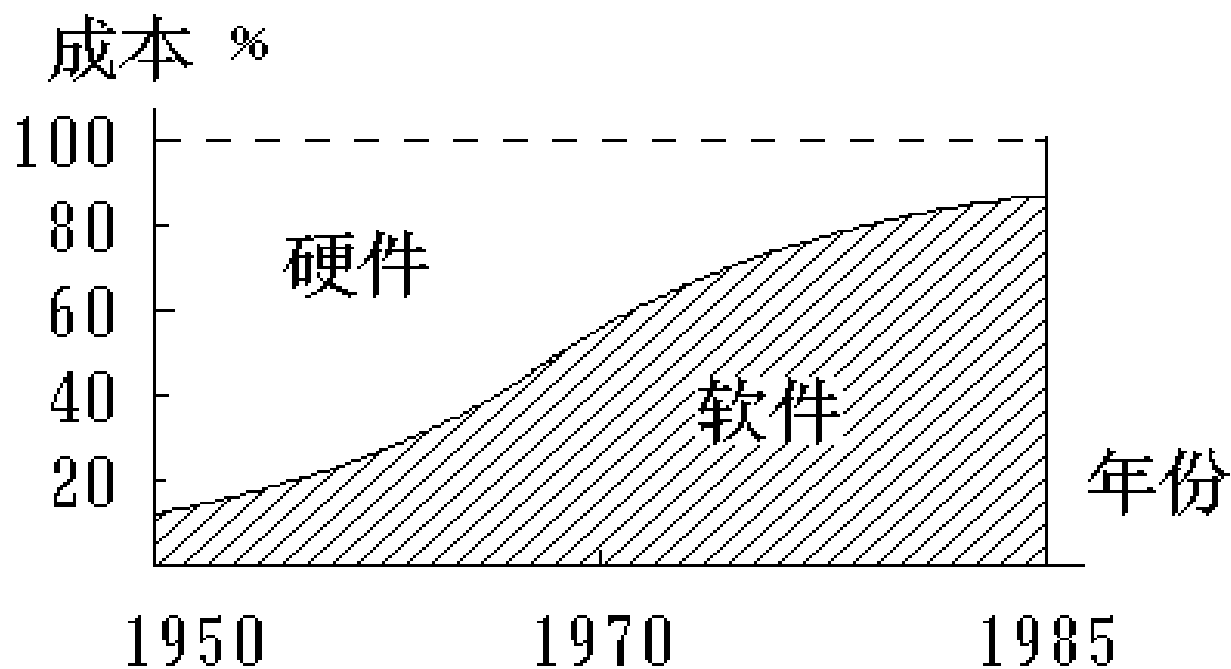
软件开发成本昂贵

- 软件的开发至今尚未完全摆脱手工艺的开发方式。软件成本相当昂贵，软件开发需要投入大量的、高强度的脑力劳动，人员成本很高，开发人员的工资大于运维等部门。

两面性

- 原因1：软件本身是复杂的
 - 实际问题的复杂性
 - 程序逻辑结构的复杂性
- 原因2：人际关系等非技术原因更复杂

软件成本超过硬件



随着云计算发展，硬件成本进一步降低

软件的分类

1. 按软件的功能进行划分:

- 系统软件
- 支持软件
- 应用软件

软件的分類

- **系统软件**

控制和协调计算机及外部设备，支持应用软件开发和运行的系统

- 操作系统
- 数据库管理系统
- 设备驱动程序
- 通信处理程序等

软件的分类

● 支持软件

支撑各种软件的开发与维护的软件，又称为软件开发环境

- 文本编辑程序
- 文件格式化程序
- 程序库系统
- 支持需求分析、设计、实现、测试和支持管理的软件

软件的分类

● 应用软件

满足用户不同领域、不同问题的应用

- 商业数据处理软件
- 工程与科学计算软件
- 计算机辅助设计 / 制造软件
- 系统仿真软件
- 智能产品嵌入软件
- 医疗、制药软件
- 事务管理、办公自动化软件
- 计算机辅助教学软件

软件的分类

1. 基于软件规模进行划分:

软件规模	投入人数	开发时间	源程序行数	特征
微型		1~4周	500	不必有严格的设计和测试文档
小型	1~2	1~6月	2000	通常没有与其他程序的接口
中型	3~5	1~2年	0.5-5万	需要有严格的文档和设计规范
大型	5-20	2~3年	5~10万	需要按照软件工程方法进行管理
超大型	100-1000	4-5年	100万左右	必须按照软件工程开发, 有严格的质量管理措施
巨型	2000-5000	5~10年	1000万左右	同上

软件的分类

1. 基于软件工作方式划分:

- 实时处理软件
- 分时软件
- 交互式软件
- 批处理软件

软件的分类

- **按软件服务对象的范围划分：**
 - 项目软件
 - 产品软件
- **按使用的频度进行划分：**
 - 一次使用
 - 频繁使用
- **按软件失效的影响进行划分：**
 - 高可靠性软件
 - 一般可靠性软件

软件的种类如此众多，软件的功能这么强大，软件是如何发展起来的呢？从最初的软件到现在的巨型系统软件，软件的发展是不是一帆风顺呢？

3 软件的发展

- 第一阶段，程序设计阶段，从1946年到20世纪60年代初。
- 第二阶段，程序系统阶段，从20世纪60年代初到20世纪70年代初。程序的规模已经发展的很大，需要多人分工协作，小组模式。
- 第三阶段，软件工程阶段，从20世纪70年代中期至80年代中期。在这个阶段，软件工程师把工程化的思想加入到软件的开发过程中。
- 第四阶段，面向对象阶段， 80年代中期至今

4. 软件危机

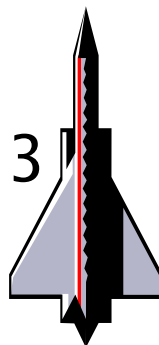
- 20 世纪60年代以前，计算机刚刚投入实际使用，软件设计往往只是为了一个特定的应用而在指定的计算机上设计，基本上是个人设计、个人使用。
- 60年代中期，大容量、高速度计算机的出现，使计算机的应用范围迅速扩大，软件开发急剧增长。高级语言开始出现；操作系统的发展引起了计算机应用方式的变化。软件系统的规模越来越大，复杂程度越来越高，软件可靠性问题也越来越突出。
- 原来的个人设计、个人使用的方式不再能满足要求，在计算机软件的开发和维护过程中遇到的一系列严重的问题。

软件危机

1963年美国飞往火星的火箭爆炸，造成1000万美元的损失。
原因是FORTRAN程序：

```
DO 5 I=1, 3
```

误写为：DO 5 I=1. 3



1967年苏联“联盟一号”载人宇宙飞船在返航时，由于软件忽略一个小数点，在进入大气层时因打不开降落伞而烧毁。

软件危机！！

60年代后期软件开发出现的问题

- ①软件开发费用和进度失控。费用超支、进度拖延的情况屡屡发生。
- ②软件的可靠性差。尽管耗费了大量的人力物力，而系统的正确性却越来越难以保证，出错率大大增加，造成的损失惊人。
- ③生产出来的软件难以维护。很多程序缺乏相应的文档资料，程序中的错误难以定位，难以改正，有时改正了已有的错误又引入新的错误
- ④用户对“已完成”的系统不满意现象经常发生。

软件危机

- IBM公司的 OS/360，共约100万条指令，花费了5000个人年；经费达数亿美元，而结果却令人沮丧，错误多达2000个以上，系统根本无法正常运行。OS/360系统的负责人Brooks这样描述开发过程的困难和混乱：“...像巨兽在泥潭中作垂死挣扎，挣扎得越猛，泥浆就沾得越多，最后没有一个野兽能够逃脱淹没在泥潭中的命运。”

软件危机的原因

(1) 随着软件应用范围的增广，软件的规模愈来愈大，复杂性也急剧地增加。缺乏统一的、规范的方法论的指导，依靠个人经验。

(2) 忽视软件开发前期的需求分析。

编程越早，需求分析不充分，完成时间越长

(3) 缺乏软件开发的经验和有关软件开发数据的积累，使得开发工作的计划很难制定。

(4) 忽视软件文档，文档常常内容不一致，有时甚至没有文档，也是造成开发效率低下的原因。

对软件开发、维护、用户都很重

要

软件危机的原因

(5) 忽视测试阶段的工作，或不负责任的测试员，提交用户的软件质量差。

“错误是不可避免的”，该如何理解这句话？

(6) 轻视软件的维护，导致软件在交付后难以维护，程序错误难以定位和排除。

2 软件工程

- 人们普遍认识到：软件开发不是个人创作，其开发过程不应该依赖个人能力，软件开发应该是一种组织良好、管理严密、各类人员协同配合、共同完成的工程项目；
- 1968 年北大西洋公约组织的计算机科学家在联邦德国召开国际会议，第一次讨论软件危机问题，并正式提出“软件工程”一词，从此一门新兴的工程学科——软件工程——为研究和克服软件危机应运而生。

解决软件危机的途径达成共识

- 1、软件开发应该是一种组织良好、管理严密、各类人员协同配合、共同完成的工程项目。思考：还需要工匠精神吗？
- 2、必须充分吸收和借鉴人类长期以来从事各种工程项目所积累的行之有效的原理、概念、技术和方法，特别是要吸取几十年来从事计算机硬件研究和开发的经验教训。
- 3、应该推广使用在实践中总结出来的开发软件的成功的技术和方法。
- 4、应该开发和使用更好的软件工具。

解决软件危机的途径达成共识

总之，为了解决软件危机，既要有**技术措施**（方法和工具），又要有必要的**组织管理措施**。

- 技术措施
- 管理措施

按工程化的原则和方法组织软件开发工作是有有效的，是摆脱软件危机的一个主要出路。**软件工程**正是从**管理**和**技术**两方面研究如何更好地开发和维护计算机软件的学科。

2.1 软件工程的定义和内容

1983年IEEE给软件工程的定义：“软件工程是开发、运行、维护和修复软件的系统方法。”强调“系统方法”不是“个人技巧”。

专家观点：

Fairly认为：“软件工程学是为了在成本限额以内按时完成开发和修复软件产品所需要的系统生产和维护技术及管理学科”。强调“成本限额以内”及“技术和管理”两个方面。

Fritz Bauer认为：“软件工程是为了经济地获得可靠的且能在实际机器上有效地运行的软件，而建立和使用的完善的工程化原则”。强调“经济地开发出高质量的软件，应建立和使用的完善的工程化原则”。

1993年IEEE进一步给出了一个更全面的定义：

软件工程是：

- (1) 把系统化的、规范的、可度量的途径应用于软件开发、运行和维护的过程，也就是把工程化应用于软件中；
- (2) 研究（1）中提到的途径。

2004 年 由 IEEE/ACM 联 合 发 布 的 Software Engineering2004报告强调了对软件工程的新定义，即软件工程是：以系统的、科学的、定量的途径，把工程应用于软件的开发、运营和维护；同时，开展对上述过程中各种方法和途径的研究。

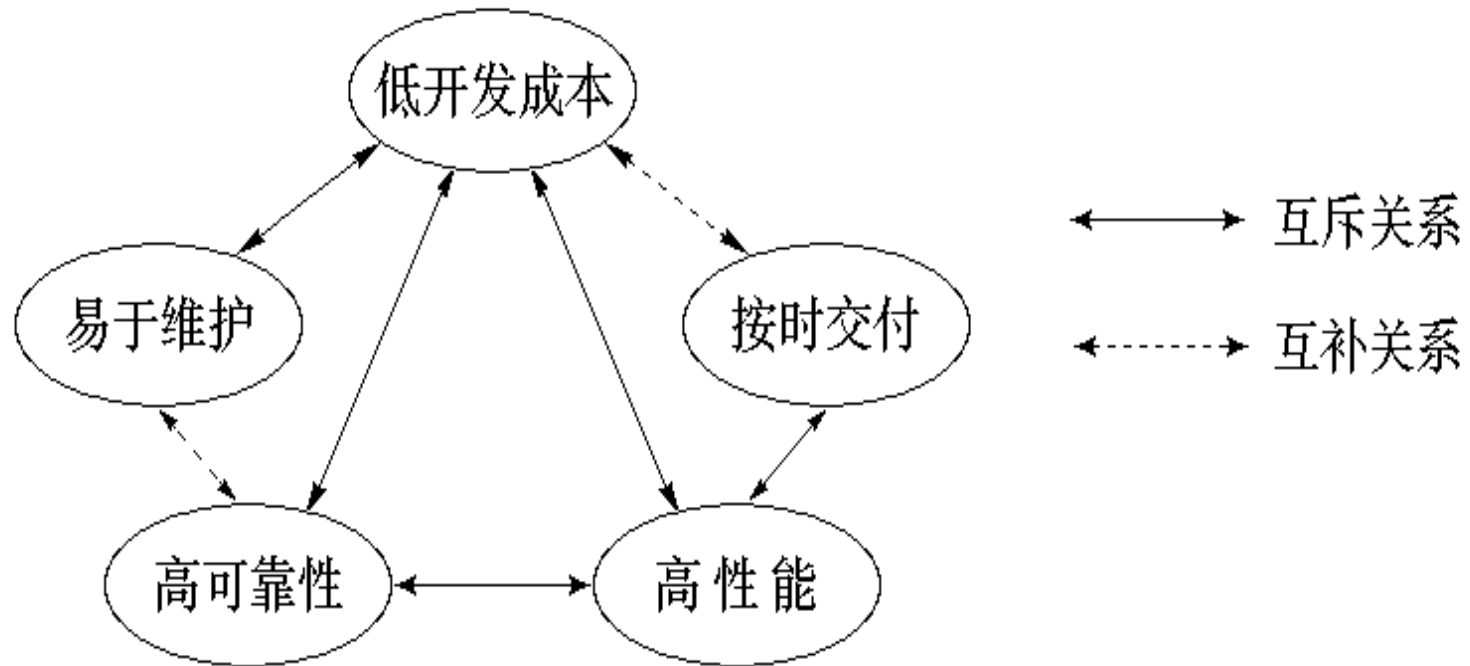
解读

软件工程是指导计算机软件开发和维护的工程学科。采用工程的概念、原理、技术和方法来开发和维护软件，把经过时间考验而证明正确的管理技术和当前能够得到的最好的技术方法结合起来，以经济地开发出高质量的软件并有效地维护它，这就是软件工程。

软件工程的目标

- 付出较低的开发成本;
- 达到要求的软件功能;
- 取得较好的软件性能;
- 开发的软件易于移植;
- 需要较低的维护费用;
- 能按时完成开发工作, 及时交付使用。

目标之间的关系



开发完美软件?

为什么不能等到软件完美之后再发布?



保时捷Macan

54.5-92.5万元



别克昂科旗

29.99-37.99万元



奔驰C级

30.78-47.48万元



大众途岳

16.58-22.38万元



大众迈腾

18.61-30.99万元

为什么还有人买“质量不是最好”的汽车呢?

课程目标

- 目标是让大家：

1. 研发出符合用户需求的软件。通过实际的工作收集、推导、提炼需求

2. 通过一定的软件流程，在预计的时间内发布“足够好”的。经历了一定的软件流程，通过全体团队成员的努力，在一个长期阶段内逐步完成的。

3. 能证明所开发的软件是可以维护和继续发展的。

对用户需求的分析有详细的文档说明，主要功能的设计文档说明和软件的实际行为一致；源代码完整并能构建出符合质量要求的版本；关键模块有可以正常执行的单元测试、压力测试脚本，等等。

2.2 工程化思想

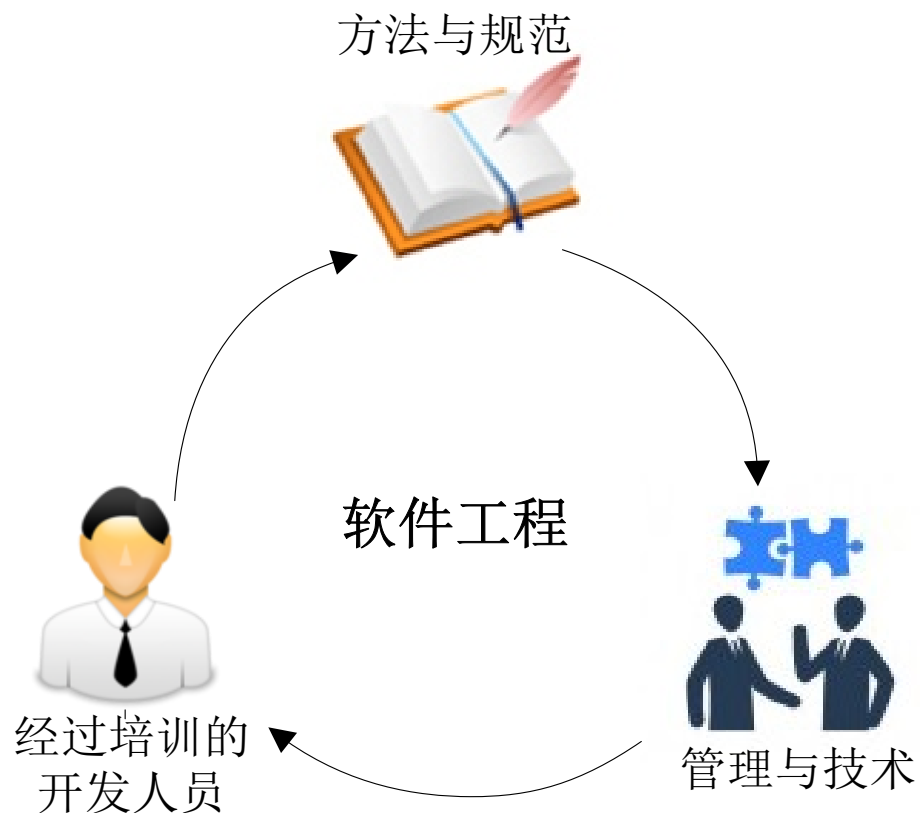
- 到底什么是工程化呢？
- 类比我们较熟悉的建筑工程，盖一个大楼
- IEEE定义：工程化是把系统的、受训到、可以计量的方法应用到结构、机械、产品、系统或处理过程。

目标

- 将软件开发工程化的目标是通过规范软件开发的步骤和每一步的要求，使软件开发像在流水线上生产机械零件一样，开发进度不依赖于个人的能力，并且开发的软件是合乎要求的，质量是可以测量和保证的。
- 一线互联网公司，比如硅谷的 Google、Facebook、Amazon，比如中国的阿里巴巴、百度、腾讯等，其软件开发已经形成体系的流水线式作业。

目标

- 形成一个能有效控制软件开发质量的运行机制



2.3 软件工程的基本原理

1. 用分阶段的生存周期计划严格管理开发过程。

软件开发周期划分成若干阶段，并相应地制定出切实可行的计划

2. 坚持进行阶段评审。

因此，软件的质量保证工作不能等到编码阶段结束之后再进行。

3. 实行严格的产品控制。

实行基准配置管理（又称变更控制），当需求变更时，其他各个阶段的文档或代码都要随之相应变化，以保证软件的一致性。

软件工程的基本原理

4. 采用现代程序设计技术。

采用先进的技术既可提高软件开发和维护的效率，又可以提高软件质量并减少维护的成本。

5. 结果应能清楚地审查。

应根据软件开发的总目标及完成期限，尽量明确地规定开发小组的责任和产品标准。

6. 开发小组的人员应少而精。

减少交流时间

7. 承认不断改进软件工程实践的必要性。

3 软件安全

- 2001年才出现软件安全方面的著作，此时计算机科学家才开始系统地思考如何构建安全的软件。
- 软件安全领域的权威专家Gary McGraw博士认为，软件安全就是使软件在受到恶意攻击的情形下依然能够继续正确运行的工程化软件思想。

软件安全开发重要性

- 丰田汽车“暴冲”事件。2007年一辆凯美瑞暴冲致一死一伤。数百位丰田汽车车主声称车辆发生无预警加速暴冲状况。
- 嵌入式系统专家检查了丰田电子节流阀系统软件原始码，在其中发现缺陷，所包含的错误码就是导致车辆暴冲事故的原因。

丰田的软件缺陷分为三类：

1、非常业余的结构设计

- 软件设计的基本要求是模块尽量简单化。但丰田的工程师显然没有遵循这原则。有67条函数复杂度超过50，意味着运行这个函数可能出现超过50种不同的执行结果，属于“非可测”级别。12条复杂度超过100，达到“非可维护”级别。最复杂的一条函数有超过1300行代码，146个可能执行路径——正好用于根据各传感器数值计算节气门开关角度。
- 极复杂的代码带来的是极复杂的功能。
- 丰田的软件包含了超过一万一千个全局变量。

丰田的软件缺陷分为三类：

2、不符合软件开发规范。

- 汽车相关软件开发人员编程规范，称为MISRA C。
- MISRA C拥有超过100条规范， Barr对照结果是丰田的程序拥有超过80000处违规代码。根据统计，违规数量可以用于预测代码中暗藏的缺陷（Bug）数量。
- MISRA C其中一个规则的内容是不得使用递归。
- 递归的问题在于其嵌套层数无法预测，从而导致可能消耗的存储器容量无法控制。
- 丰田辩称他们遵循的不是行业规范，而是丰田内部编程规范。但检查发现丰田的代码甚至没有遵循丰田内部规范。

丰田的软件缺陷分为三类：

3、对关键变量缺乏保护

- 嵌入式系统，或者任何系统，都会在一定条件下发生硬件或者软件错误。汽车作为一个时常在震动、发热、位移的系统，它的内部环境不能说不恶劣，发生硬件错误的可能性甚至更高。
- 由于某些不可抗原因，一个电平从高变成低，或者反过来，那么这个变量就被更改了。这被称为“**位反转 (Bit Flip)**”。为了对抗这样的事情发生，需要对变量进行保护。保护的方法一般是镜像法。
- 另一个保护措施是错误校验码。在2005年款凯美瑞的系统中完全没有使用，丰田却告诉NASA他们用了。

3.1 软件安全与软件缺陷

- 软件安全是计算机系统安全的核心和关键。实际上，众多的网络安全事件是因为软件自身存在诸多的问题，从而才能被恶意软件或黑客所利用。
- 有时由于发布时间问题，程序员明明知道软件存在一些缺陷，但仍然会发布。
- 美国联邦健康保险交换系统HealthCare.gov自2013年10月1日开通运营以来一直遭受各种问题的困扰，该网站的承包商表示，他们仅有两周的时间来测试该系统，实际上需要几个月的时间才可以完成，因此，网站崩溃的原因之一便是测试时间太短，程序中存在大量的软件漏洞。
- 一些黑客甚至能攻入软件公司，直接在源程序中植入恶意代码。

3.2 软件安全威胁

- 1. 软件缺陷
- IEEE729-1983对缺陷有一个标准的定义：从产品内部看，缺陷是软件产品开发或维护过程中存在的错误、毛病等各种问题；从产品外部看，缺陷是系统所需要实现的某种功能的失效或违背。

软件安全漏洞持续增长的原因

- (1) 软件开发安全意识淡薄，很少从“攻击者”的角度来思考软件安全问题。
- (2) 软件开发缺乏安全知识，软件开发人员很少进行安全能力与意识的培训。
- (3) 软件趋向大型化，第三方扩展增多

软件面临的外部威胁

- 恶意软件增多。计算机病毒、蠕虫、特洛伊木马、后门、僵尸、间谍软件等。
- (1) 破坏正常软件运行
- (2) 窃取重要数据
- (3) 监视用户行为
- (4) 控制目标系统
- (5) 加密文件索要赎金

解决软件安全

- 解决软件安全问题的根本方法就是改善我们建造软件的方式，以建造健壮的软件，使其在遭受恶意攻击时依然能够安全可靠和正确运行。
- McGraw博士提出软件安全工程化的三个支柱：风险管理、软件安全切入点、安全知识。
 - 风险管理是一种贯穿软件开发生命周期的战略性方法；
 - 软件安全切入点是在软件开发生命周期中保障软件安全的一套最佳或良好实际操作方法，其中包括：滥用案例、安全需求、体系结构风险分析、代码审核、基于风险的安全测试、渗透测试和安全操作。